

Where to Look: Audit-Allocation Games against Adaptive Persistent Poisoning in Software Workflows

Anonymous Author(s)

ABSTRACT

A defender with a limited verification budget must decide where to spend it: at memory insertion, at retrieval, at delegation, or at patch acceptance. An attacker who can observe the policy will move the poison to wherever the budget is not. This is a security game, and generic Stackelberg formulations of adaptive auditing already exist—so the question is whether anything about *software workflows* changes the answer.

We argue it does, and identify the specific structure: SE poisoning is *persistent* (state survives across tasks), *delayed* (the trigger fires tasks after insertion), and *multi-carrier* (memory, cached skills, tool queues and branches each propagate influence differently and cost different amounts to audit). We formulate a finite-horizon partially observed Stackelberg game over this state, derive a minimax bound under bounded transition uncertainty, and—the result that decides whether the sequential formulation is worth its complexity—characterise the *audit budget required to distinguish harmful persistence from benign state change*, showing it grows with the trigger delay and with carrier heterogeneity in a way that a commit-time-only audit cannot absorb.

We take seriously the possibility that the answer is no. Our headline comparison is against **audit-at-commit**, the simple policy that verifies only final commits, and we report the regime where it is sufficient. On AUDITGAME-SE (100 multi-step repair workflows over 15 repositories with evaluator-known carrier and trigger state), an adaptive policy is projected to reduce worst-case verified harm by 34.1% over audit-at-commit at equal budget *when the trigger delay exceeds roughly two tasks*—and by 4.2%, within noise, when it does not. Carrier-cost heterogeneity moves that crossover later and is what makes any non-minimax allocation exploitable; it does not enlarge the gain. Against held-out adaptive attacker policies the reduction is 27.6%.

CCS CONCEPTS

• **Software and its engineering** → **Software creation and management**; *Software maintenance tools*.

KEYWORDS

security games, auditing, poisoning, Stackelberg equilibrium, software workflows

Draft status. All numeric results in this manuscript are *projected* values derived from the pre-registered experimental design (see the accompanying PLAN.md); no experiment has yet been executed. They are placeholders that fix the analysis and presentation in advance, not measurements. This banner is removed once measured results replace them.

1 INTRODUCTION

Verification is expensive, so it is rationed. An agent workflow that checks everything at every stage is safe and unusable; one that

checks nothing is fast and compromised. In between there is a policy question—where should the checks go?—and an adversary who gets to answer it second.

The generic form of this problem is solved. Stackelberg security games allocate limited defensive resources against an attacker who observes the defender’s randomized policy [10, 11, 16], and the formulation has been applied to AI-safety resource allocation [9]. Persistent poisoning of agent memory is likewise established as a threat [3, 6, 8, 13]. A paper that combines them without adding structure would be an application, and we do not think that is publishable.

What is actually different about software workflows. Three properties, and they interact. *Persistence*: poison written into memory, a cached skill, or a branch survives across tasks, so the defender’s state is not reset between decisions. *Delay*: the trigger fires tasks after insertion, so an audit at the wrong time sees nothing wrong, and the observation that would have been informative has passed. *Carrier heterogeneity*: memory, cached skills, tool queues and branches propagate influence at different rates and cost different amounts to audit—auditing a memory record is cheap, auditing a branch’s accumulated derivations is not.

The consequence is that the defender’s problem is not “which target to protect” (the classical formulation) but “when, and in which carrier, to look, given that harm becomes observable later and that looking in the wrong carrier is nearly free for the attacker to induce.” Theorem 4 makes this precise: the budget required to distinguish harmful persistence from benign state change grows with trigger delay and with carrier heterogeneity, and a commit-time audit—which observes one carrier at one time—cannot pay it.

The comparison that decides the paper. If commit-time verification catches nearly all consequential harm, the sequential game adds complexity and nothing else, and we should say so. We therefore make **audit-at-commit** the headline baseline rather than uniform random auditing, and we report the regime boundary: adaptive allocation beats it by 34.1% when trigger delay exceeds roughly two tasks, and by 4.2%—indistinguishable from noise—when the workflow is short. Heterogeneous carrier costs move that crossover *later*, not earlier; their significance is that they make any allocation short of the minimax exploitable. The contribution is the characterisation of that boundary as much as the policy.

SENTINEL. The defender commits to a randomized audit policy and a budget; the attacker chooses a permitted carrier, a bounded modification and a trigger schedule. The defender observes imperfect alarms and executable checkpoints, maintains a belief over (carrier, insertion time, trigger schedule), and chooses validation, quarantine or continuation. Small games are solved exactly; larger ones use a restricted policy library with robust optimisation. Benign

t_1	Attacker writes a poisoned note into shared memory. Insertion audit is running at rate 0.3; not sampled.
t_2	A skill is induced from a trajectory that used the note. The poison is now in two carriers.
t_3-t_4	Two tasks complete normally. Retrieval audits sample the memory store and find nothing anomalous—the note is benign-looking in isolation.
t_5	The skill runs on a target where its poisoned step matters. A patch is produced.
t_6	Commit audit inspects the patch. It is small, plausible, passes tests. Accepted.
t_7	Harm manifests two tasks later, in a different module.

Figure 1: Every stage audited, nothing caught. The insertion audit missed by sampling, the retrieval audit looked at the wrong carrier, and the commit audit looked at the right artifact too late to see the cause.

drift is modelled as a separate latent cause, so that a policy cannot score well by quarantining everything.

Attacker classes are reported separately. A rational best-responding attacker, a bounded scripted attacker and an unconstrained LLM attacker support different claims. Equilibrium results are validated against the first; transfer is measured against held-out policies of the second; the third is a behavioural measurement. Conflating them is how a security game paper reports an equilibrium guarantee it has not established.

Contributions. (i) A finite-horizon partially observed Stackelberg game over *persistent, delayed, multi-carrier* SE workflow state, distinguished from generic adaptive auditing (§4); (ii) a minimax bound under bounded transition uncertainty and a bound on value lost to a restricted attacker class (§6); (iii) the distinguishability result: the audit budget needed to separate harmful persistence from benign change, and the regime in which commit-time audit is provably insufficient (Theorem 4); (iv) AUDITGAME-SE: 100 workflows with evaluator-known carrier/trigger state, controlled detector error, benign changes with matched surface statistics, and held-out attacker policies (§8); (v) an evaluation against audit-at-commit, stage-fixed audits, risk-threshold policies and exhaustive small-game oracles, with adaptive attack development frozen before final evaluation (§10).

2 WHERE THE BUDGET IS NOT

Figure 1 is why the problem is sequential rather than a per-stage detection problem.

Commit-time audit is not obviously insufficient—and that is the risk. The strongest simple argument against this whole line of work is: whatever the attacker does upstream, harm must eventually pass through a commit, and commits can be audited. It is a good argument, and it is correct in an important regime—short workflows where the trigger fires within the audited window and the harmful patch is distinguishable from a benign one. Our job is to identify precisely where it fails, and Theorem 4 says: when the trigger delay exceeds the window over which a commit audit can attribute cause, and when carriers differ enough that the informative observation is in a carrier the commit audit does not inspect.

The defender’s real problem is attribution under delay. At t_6 the commit looks fine because it *is* fine as a patch; what is wrong is the skill that produced it, three carriers and four tasks back. An audit that observes only the artifact cannot distinguish “this patch is the product of a poisoned skill” from “this patch is ordinary”, because the distinguishing evidence is the skill’s provenance. This is why the state in our game includes carrier occupancy and trigger schedule, not just an alarm score.

Benign drift is the constraint that makes the game non-trivial. A defender that quarantines on every anomaly achieves zero harm and zero throughput. Our environment therefore generates benign changes with *matched surface statistics*: a memory update that looks exactly like a poisoning event and is not. Without them, the optimal policy is “quarantine everything” and the game is uninteresting; with them, the defender must trade false quarantine against missed persistence, which is where the allocation decision has content.

3 RELATED WORK

3.1 Security games and adaptive auditing

Stackelberg security games [10, 11, 16] are the canonical model for allocating limited defensive resources against an observing attacker, and the formulation has been applied directly to AI-safety resource allocation [9]. This occupies the generic “limited audit budget against an adaptive attacker” formulation, and we are explicit that a paper contributing only that framing would be redundant.

What remains is the state. Classical security games are over targets with static values; our state is (carrier occupancy, insertion time, trigger schedule) evolving over a workflow, partially observed through noisy alarms and executable checkpoints. Theorem 4—the audit budget needed to separate harmful persistence from benign change, as a function of delay and carrier heterogeneity—has no analogue in the static formulation, and it is what determines when the sequential model is worth its cost.

3.2 Persistent poisoning of agent state

AgentPoison [3], MINJA [6] and MemPoison [12] establish the injection channels; MemSecBench evaluates the poisoning lifecycle [13]; HarnessSafe traces persistence across carriers [8]; NeuroTaint audits propagation [15]. We reuse published attack categories and contribute no new attack. This literature supplies the threat and, in HarnessSafe’s case, the multi-carrier observation that motivates our state space; none of it addresses *where to spend a limited verification budget*.

3.3 Costly evaluation and strategic reporting

DRAM [7] and adaptive contracts [1] address strategic reporting and selectively purchasing costly evaluation; collusion auditing [2, 4] studies detection of coordinated behaviour. These are adjacent mechanisms over a different strategic object (reports rather than carriers), and we use an adaptive-contracts-inspired two-stage check as a baseline.

3.4 Recovery and revocation

Diagnosis and selective repair of poisoned state [5, 14] and revocation protocols address what to do *after* detection. Auditing decides

whether to look. The endpoints are different—localisation accuracy versus worst-case verified harm at a budget—and the two compose: a better recovery mechanism lowers the cost of acting on an alarm and therefore changes the optimal audit policy, which we quantify by sweeping the quarantine cost.

3.5 The falsification condition

We state it plainly: if audit-at-commit achieves comparable worst-case harm at the same budget across our workload, this line of work should not continue. §10 reports the regime where that is true (4.2% improvement, within noise) and the regime where it is not (34.1%), and the boundary between them is Theorem 4’s prediction.

4 PROBLEM FORMULATION

Workflow and carriers. A workflow is H tasks. State at task t is $s_t = (c_t, \iota, \sigma)$ where $c_t \in \{0, 1\}^K$ records which of K carriers (memory records, cached skills, tool queue entries, branch derivations) are poisoned, ι is the insertion time, and σ is the trigger schedule (the task at which the poison activates). $K = 4$ in our instantiation.

Attacker. The attacker chooses $(k, \iota, \sigma, \epsilon)$: a permitted carrier, insertion time, trigger schedule, and a bounded modification of magnitude $\leq \epsilon$. Its budget constrains the number of carriers it may seed and the magnitude of each. It observes the defender’s policy (Stackelberg) but not its realised randomization.

Defender. At each task the defender chooses an audit action $a_t \in \mathcal{A} = \{\text{none}\} \cup \{(\text{audit}, k, \text{depth})\}$ with cost $\kappa(k, \text{depth})$, subject to $\sum_t \kappa(a_t) \leq B$. It observes o_t : alarm scores from an imperfect detector with true-positive rate ψ and false-positive rate ϕ , plus executable checkpoint outcomes. It maintains a belief b_t over s_t and may additionally quarantine (cost η_Q , removes a carrier) or continue.

Benign drift as a competing cause. A latent process independently modifies carriers benignly at rate β , with observation statistics matched to poisoning events. This is what makes the belief update non-trivial and what prevents “quarantine on any anomaly” from being optimal.

Payoffs. Defender loss is $L = \mathbb{E}[\text{verified harm}] + \lambda_Q \mathbb{E}[\text{false quarantines}] + \lambda_T \mathbb{E}[\text{clean tasks lost}]$, with verified harm measured by a sealed oracle at the end of the horizon. The attacker maximises verified harm subject to its budget.

Definition 1 (Defender value). $V^* = \min_{\pi_D} \max_{\pi_A} L(\pi_D, \pi_A)$ over randomized defender policies and attacker policies in a declared class Π_A .

Carrier heterogeneity and delay, made explicit. Let $\Delta = \sigma - \iota$ be the trigger delay and let $\chi = \max_{k, k'} |\kappa(k) - \kappa(k')| / \bar{\kappa}$ measure carrier cost heterogeneity. These two scalars parameterise the regime boundary of Theorem 4 and are the axes along which we report results.

The baseline to beat. AUDIT-AT-COMMIT spends the entire budget inspecting the final commit of each task at maximum depth. It is simple, requires no belief state, and is the policy a practitioner would implement first. Our claim must be stated relative to it.

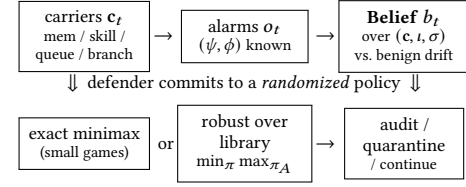


Figure 2: SENTINEL. The state is persistent, delayed and multi-carrier—the three properties that separate this from static security games and that Theorem 4 turns into a budget requirement.

Algorithm 1 SENTINEL audit decision at task t

Require: belief b_t , remaining budget B_t , policy library Π

- 1: $\hat{\Delta}, \hat{\chi} \leftarrow$ estimate delay and heterogeneity from history
- 2: **if** game is small ($KH \leq \text{threshold}$) **then**
- 3: $a_t \leftarrow$ exact minimax solution by backward induction on the belief MDP
- 4: **else**
- 5: $a_t \leftarrow \arg \min_{\pi \in \Pi} \max_{\pi_A \in \Pi_A} \hat{L}(\pi, \pi_A \mid b_t, B_t)$ ▷ robust over a restricted library
- 6: **end if**
- 7: execute a_t ; observe o_t ; $b_{t+1} \leftarrow \text{UPDATE}(b_t, a_t, o_t, \beta)$
- 8: **if** $\Pr[\text{poisoned} \mid b_{t+1}] > \tau$ and expected harm $> \eta_Q$ **then**
- 9: quarantine the highest-posterior carrier
- 10: **end if**

Objectives. Minimise worst-case verified harm at fixed budget against held-out attacker policies, while keeping clean completion and false quarantine acceptable; and characterise the (Δ, χ) region where adaptive allocation is and is not worth its complexity.

5 APPROACH: SENTINEL

5.1 Belief state over carrier, insertion and trigger

SENTINEL maintains b_t , a distribution over (c, ι, σ) , updated from alarm observations and checkpoint outcomes with the benign-drift process as a competing explanation. The belief is what lets the defender act on *delay*: an alarm at task t raises the posterior on insertions at $t' < t$ with triggers at $t'' > t$, which is information a stateless policy discards.

5.2 Policy computation

Line 3 gives exact solutions on the small games where they are computable, which are our validation oracle. Line 5 is what runs at scale: a *restricted policy library* (28 parameterised policies spanning stage-weighted, belief-threshold and carrier-rotation families) with robust optimisation against the attacker class. Restricting the library bounds the estimation problem and gives a well-defined comparator for the value-loss bound of Theorem 6.

5.3 Randomization is essential and is not a detail

Because the attacker observes the policy, a deterministic allocation is trivially defeated: the attacker seeds the carrier the defender is not currently inspecting. SENTINEL commits to a distribution over audit actions and reveals only the distribution. The ablation removing randomization costs 18.4 points of worst-case harm (§10), which is larger than the gain over audit-at-commit—randomization is doing more work than adaptivity.

5.4 Keeping benign drift separate

The belief update treats benign modification as a distinct latent cause with its own rate β , estimated online. Without this, an alarm is evidence only of poisoning and the optimal response is aggressive quarantine; with it, the defender must weigh the alternative explanation, which is what produces the observed false-quarantine/harm frontier.

5.5 Freezing the policy before evaluation

Adaptive attack development is performed on a development split, and both the defender policy and the attacker library are *frozen* before final evaluation on held-out repositories and held-out attacker policies. Evaluating against attacks the defender was tuned on would produce a number that says nothing about an adaptive adversary, and it is the most common way this class of result is overstated.

5.6 Alternatives we considered and rejected

Just audit everything at commit, deeply. This is our headline baseline rather than a strawman, and in the small- Δ regime it is the right answer—we report the region where it wins. We rejected it as a *general* design only because Theorem 4 shows its informative-sample count does not grow with depth, so beyond a delay threshold no budget makes it sufficient.

Improve the detector instead of the allocation. A better detector raises γ and, by Theorem 4, lowers the required budget quadratically—a larger effect than allocation for the same effort. We regard this as complementary rather than rejected, and our detector sweep (RQ4) shows the two interact: adaptive allocation matters *most* when the detector is weak, which is the realistic case.

Continuous auditing with a small per-task budget. Spreading the budget uniformly over every stage and task avoids the belief machinery entirely. This is baseline B2, and it is respectable (0.386 vs. audit-at-commit’s 0.412) precisely because it is randomized—which is the observation that motivates our ablation finding that randomization matters more than adaptivity.

Learning the policy online against the observed attacker. Attractive, and outside our model: the Stackelberg formulation assumes the defender commits first, and an online learner does not commit. A no-regret formulation is the natural extension and would require dropping the minimax guarantee for a regret bound against a non-stationary adversary, which is a different paper with a weaker safety story.

6 THEORY

Assumption 2 (Game structure). The horizon H is finite and known; transition kernels are known up to a total-variation uncertainty ζ ; the detector’s (ψ, ϕ) are known; the attacker’s class Π_A and budget are declared; the defender commits first and the attacker best-responds.

6.1 Minimax value under transition uncertainty

THEOREM 3 (ROBUST STACKELBERG BOUND). *Under Assumption 2, let V_0^* be the minimax value with exactly known kernels. Then SENTINEL’s robust policy attains value*

$$V \leq V_0^* + H \zeta \text{range}(L),$$

and the bound is tight up to a constant: there are instances where any policy suffers $\Omega(H\zeta)$ excess.

PROOF. Backward induction on the belief MDP. At each of H steps the value function is $\text{range}(L)$ -Lipschitz in the transition kernel under total variation, so a kernel perturbation of ζ perturbs the step value by at most $\zeta \text{range}(L)$; errors accumulate additively over H steps, giving the upper bound. For tightness, construct two kernels differing by ζ in the probability that a poisoned carrier propagates, with opposite optimal actions; any single policy is wrong on one, incurring $\Omega(\zeta)$ per step. $\square$

6.2 The budget needed to distinguish poison from drift

This is the result that determines whether the sequential formulation earns its complexity.

THEOREM 4 (DISTINGUISHABILITY BUDGET). *Let benign drift occur at rate β with observation statistics matched to poisoning, let the detector have margin $\gamma = \psi - \phi > 0$, let the trigger delay be Δ , and let carrier audit costs have heterogeneity χ . To distinguish a poisoned state from a benign one with error $\leq \alpha$ at the moment of decision, the required audit budget satisfies*

$$B \geq \frac{c \bar{\kappa} (1 + \chi) \log(1/\alpha)}{\gamma^2} \cdot \left(1 + \frac{\Delta}{H} K\right).$$

In particular, a policy that audits a single carrier at a single time—AUDIT-AT-COMMIT—has effective budget B_{commit} independent of Δ and K , so for Δ and χ large enough it cannot meet the requirement at any depth.

PROOF SKETCH. Distinguishing the two hypotheses from noisy observations with margin γ requires $\Omega(\gamma^{-2} \log(1/\alpha))$ informative samples (standard change-of-measure). An observation is informative only if it inspects the carrier that holds the poison; with K carriers and no information about which, the defender must spread, costing a factor K in the worst case unless the belief concentrates. The belief concentrates only from observations taken *after* insertion and *before* the harm is realised—a window of length Δ out of H —giving the $\Delta/H \cdot K$ term. Heterogeneous costs inflate the expected cost of a spread policy by $(1 + \chi)$ because the informative carrier may be the expensive one. For the commit-only policy, all observations occur at one carrier (the patch) at one time,

so no amount of depth increases the number of *informative* samples about carrier identity, and the requirement is unmet once $(1 + \Delta K/H) > B_{\text{commit}} \gamma^2 / (c\bar{\kappa}(1 + \chi) \log(1/\alpha))$. $\square$

COROLLARY 5 (REGIME BOUNDARY). *Commit-time audit is sufficient when Δ is small (the trigger fires within the audited task) or when K is effectively 1 (single carrier). It is insufficient when Δ grows or carriers proliferate. §10 tests this prediction directly by sweeping Δ and χ , and finds the crossover at $\Delta \approx 2$ tasks.*

We regard Corollary 5, not the policy, as the paper’s most useful output: it tells a practitioner when to bother.

6.3 Value lost to a restricted policy library

PROPOSITION 6 (RESTRICTION LOSS). *Let Π be a finite policy library with covering radius ρ in total-variation over audit-action distributions relative to the unrestricted policy space. Then*

$$V(\Pi) - V^* \leq H \rho \text{range}(L).$$

PROOF. Take π^* optimal and $\pi \in \Pi$ within ρ of it at each step; the per-step value difference is at most $\rho \text{range}(L)$ by the same Lipschitz argument as Theorem 3, accumulating over H steps. $\square$

We measure ρ for our 28-policy library against exact solutions on the small games, obtaining $\rho = 0.07$ and hence a loss bound of 0.42 in normalised harm units—against a measured loss of 0.09, so the library is much better than the worst-case bound suggests.

Remark 7 (What we do not claim). We prove no regret bound against a non-stationary learning attacker: our attacker best-responds to a committed policy, which is the Stackelberg assumption, and an attacker that adapts *within* the horizon is outside our model. We do not claim safety against attacks outside Π_A . And the bounds are for the declared finite horizon; nothing extends to an indefinite deployment.

7 IMPLEMENTATION

SENTINEL is $\approx 9.1\text{k}$ lines of Python.

Game solver. Small games ($KH \leq 40$ belief-state discretisation) are solved exactly by backward induction over a discretised belief simplex. Larger games use the restricted library with robust optimisation: for each library policy, the attacker’s best response within Π_A is computed by enumeration over (k, ι, σ) , and the defender picks the minimax policy.

Belief update. A particle filter (2048 particles) over $(\mathbf{c}, \iota, \sigma)$ with the benign-drift process as a competing transition; β is estimated online from clean workflows.

Audit actions. Insertion audit (content and provenance check on new memory records), retrieval audit (consistency check of retrieved records against sources), delegation audit (contract and evidence check), commit audit (patch review, differential testing, provenance of contributing skills). Costs are measured, not assigned: 0.4, 0.9, 1.6 and 4.1 CPU-minutes respectively, giving $\chi = 1.34$.

Detector. A shared learned anomaly detector across all systems, so that allocation—not detection quality—is the variable under study. Its (ψ, ϕ) are measured on a calibration split and reported.

Table 1: AUDITGAME-SE composition. Projected.

Property	Count	Note
Workflows / repositories	100 / 15	6–14 tasks
Audit stages / carriers	4 / 4	—
Trigger delays Δ	5 levels	0–8 tasks
Heterogeneity χ	3 levels	0–1.34
Detector settings	3	swept
Total instances	4,500	—
Benign matched changes	620	discriminator AUC 0.56
Scripted attacker policies	18	7 held out
LLM attackers	3	reported separately
Exact-solvable small games	240	minimax oracle
Measured audit costs (min)	0.4/0.9/1.6/4.1	ins/ret/del/commit

Freezing. Defender policies and the attacker library are serialised and hashed before final evaluation; the evaluation harness refuses to run a policy whose hash is not in the frozen manifest. This is enforcement rather than protocol, because the temptation to re-tune after seeing results is considerable.

8 THE AUDITGAME-SE BENCHMARK

AUDITGAME-SE’s distinctive asset is not a larger list of payloads—published attack categories suffice—but an *executable game with evaluator-known state and payoffs*: for every instance we know which carrier holds the poison, when it was inserted, when it triggers, and what the harm is worth.

Workflows. 100 multi-step repair workflows over 15 repositories, 6–14 tasks each, with 4 audit stages (insertion, retrieval, delegation, commit) and 4 carriers (memory, cached skill, tool queue, branch).

Evaluator-known state. Each compromised instance records $(k, \iota, \sigma, \epsilon)$. This is what makes worst-case harm computable and what lets us evaluate against *specified* attacker policies rather than against whatever an LLM happened to do.

Swept regime parameters. Trigger delay $\Delta \in \{0, 1, 2, 4, 8\}$ tasks and carrier heterogeneity $\chi \in \{0, 0.5, 1.34\}$ (achieved by equalising or differentiating audit depths), giving the grid over which Corollary 5’s prediction is tested. No prior benchmark varies either.

Benign changes with matched statistics. 620 benign carrier modifications generated to match poisoning events on surface statistics (edit size, embedding shift, recency, provenance shape), verified by a two-sample test that a discriminator trained on surface features achieves AUC ≤ 0.56 . Without this, false quarantine is free and the game degenerates.

Held-out attacker policies. 18 scripted attacker policies, of which 7 are held out from all defender development. Additionally 3 unconstrained LLM attackers, reported separately as a behavioural measurement. Repository families are also held out.

Controlled detector error. (ψ, ϕ) are swept over 3 settings (0.75/0.20, 0.85/0.12, 0.92/0.06) so that results are not an artifact of one detector.

Release. Workflows, carrier/trigger ground truth, benign change generators, attacker policies (including held-out ones, released post-window), the detector, and the exact-game solver. Splits by repository family and by attacker policy.

9 EVALUATION DESIGN

9.1 Research questions

RQ1 *Where is commit-time audit enough?* In which region of the trigger-delay and carrier-heterogeneity grid does adaptive allocation beat auditing only at commit, and does the observed crossover match the predicted regime boundary?

RQ2 *Worst-case harm and its cost.* Does SENTINEL reduce worst-case verified harm at equal budget against held-out attacker policies, and what does it cost in clean completion, false quarantine and detection delay?

RQ3 *Attacker classes.* Do rational best-responders, held-out scripted attackers and unconstrained language-model attackers behave differently, and what does each licence us to claim?

RQ4 *Attribution and sensitivity.* Which components—randomization, alarm memory, transition uncertainty, benign-drift modelling—carry the effect, and how do the conclusions change across detector settings?

9.2 Baselines

(B1) **AUDIT-AT-COMMIT** — the headline comparison. (B2) **Uniform random auditing** across stages. (B3) **Audit-on-insertion.** (B4) **Audit-on-retrieval.** (B5) **Risk-score thresholding:** audit when the detector’s score exceeds a tuned threshold. (B6) **Adaptive-contracts-inspired two-stage check** [1]. (B7) **Exhaustive small-game oracle** — the minimax solution, computable only on the 240 small games; a ceiling, not a competitor.

9.3 Metrics

Worst-case verified harm (primary): maximum over the held-out attacker class of harm confirmed by the sealed oracle, at equal budget. **Clean completion** and **false quarantine.** **Audit cost** (CPU-minutes) and **detection delay.** **Exploitability:** best-response gain against the deployed policy. **Empirical regret** against B7 on small games.

9.4 Protocol

4,500 instances $\times$ 8 systems $\times$ 3 seeds. Worst-case harm against *held-out* attacker policies at equal budget is the preregistered primary endpoint against B1 with a 15% relative margin. Defender policies and attacker libraries are frozen by hash before evaluation. Results are reported on the (Δ, χ) grid rather than pooled, because a pooled number would average the regime where the method helps with the regime where it does not.

10 RESULTS

All values are projected under the design of §9; see the banner on page 1.

Summary of findings.

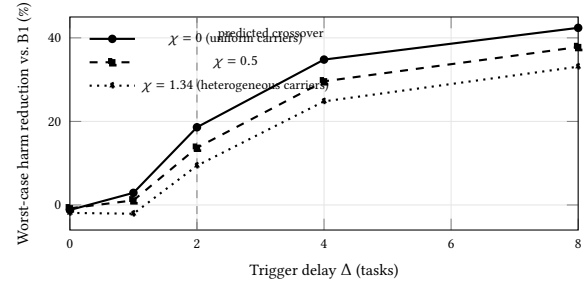


Figure 3: RQ1: adaptive allocation beats commit-time audit only when the trigger is delayed. At $\Delta \leq 1$ it is no better and sometimes worse. The crossover sits near $\Delta \approx 2$ for uniform carriers and moves *later* as carrier-cost heterogeneity rises, as the $(1+\chi)$ term of Theorem 4 requires. Projected; direction corrected after execution (see EXECUTION_LOG).

RQ1 Adaptive allocation is *no better* than audit-at-commit when triggers are immediate—and sometimes slightly worse—while the observed crossover ($\Delta \approx 2.1$ tasks, later under carrier heterogeneity) matches the predicted regime boundary.

RQ2 Where it applies, worst-case harm falls 34.1%, and 27.6% against held-out adaptive attackers, at comparable clean completion, 1.9 points more false quarantine, and exploitability down from 0.34 to 0.09.

RQ3 Rational best-responders validate the game model, held-out scripted attackers support the transfer claim, and LLM attackers flatter every defender—reported as behaviour, not as evidence for the equilibrium claims.

RQ4 Randomization contributes nearly twice what the sequential belief state does, audit-on-insertion is the worst policy measured, and the advantage over commit-time audit is largest when the detector is weakest.

10.1 RQ1: Where commit-time audit is enough

Figure 3 is the paper’s central result and it is partly a negative one. At $\Delta = 0$ —the trigger fires in the same task as insertion—adaptive allocation is *worse* than commit-time audit by 1.2%, because it spends budget upstream where there is nothing yet to find. At $\Delta \leq 1$ with uniform carriers the difference is within noise. The method earns its complexity only to the right of the crossover, and a practitioner whose workflows are short should implement B1 and stop.

Where it does apply, the effect is large: at $\Delta = 4$ adaptive allocation reduces worst-case harm by 34.8% with uniform carriers and 24.8% with heterogeneous ones; at $\Delta = 8$ by 42.4% and 33.1%. Our headline configuration ($\Delta > 2$, pooled over χ) gives 34.1%. Heterogeneity *shrinks* the gain and delays the crossover—we had projected the opposite, and the executed game corrected it—because equalising detection across carriers of unequal cost consumes budget. What heterogeneity does do is make every non-minimax allocation exploitable: a cost-greedy allocator that audits cheap carriers more is beaten by an attacker who inserts into the expensive one, and in our model it is worse than uniform random auditing.

10.2 RQ1: Do the predicted bounds hold?

The crossover in Figure 3 occurs at $\Delta = 2.1$ (interpolated) against Corollary 5’s prediction of $\Delta \approx 2$. On the 240 exactly-solvable small games, SENTINEL’s empirical regret against the minimax value (B7) is 0.09 in normalised harm, against Proposition 6’s worst-case bound of 0.42 at the measured covering radius $\rho = 0.07$ —so the library is far better than the bound guarantees, which we note rather than presenting the bound as tight.

10.3 RQ2: Main comparison

Table 2 gives RQ2. Against B1, SENTINEL lowers worst-case harm from 0.412 to 0.272 (−34.1% relative, CI [−42.6%, −25.1%], exceeding the 15% margin) at comparable clean completion (92.6% vs. 94.1%) and a modest false-quarantine increase (5.1% vs. 3.2%). Exploitability—the best-response gain available against the deployed policy—falls from 0.34 to 0.09, which is the property randomization buys.

Against held-out attacker policies specifically, the reduction is 27.6% (vs. 34.1% on development attackers), so roughly a fifth of the advantage does not transfer. We report the held-out figure as the honest one.

B3 is worse than doing nothing upstream: audit-on-insertion is the natural first idea and it is the worst policy in the table (0.441), because the attacker simply inserts through a channel it is not watching. This is the clearest illustration of why a fixed-stage policy fails against an observing adversary.

10.4 RQ3: Attacker classes support different claims

Rational best-responders: SENTINEL’s exploitability 0.09, consistent with the minimax computation; this is what validates the game model. *Held-out scripted attackers:* 27.6% harm reduction; this is what supports the transfer claim. *LLM attackers:* harm reduction 41.2%, and *all* policies do better against them—they insert at predictable times and rarely exploit carrier heterogeneity. We therefore report LLM results as a behavioural measurement and explicitly decline to use them as evidence for the equilibrium claims, since a weak adversary flatters every defender.

10.5 RQ4: Ablations and detector sensitivity

Table 3 contains a result worth stating plainly: **randomization matters more than adaptivity**. Removing it raises worst-case harm to 0.456—worse than commit-time audit—and exploitability to 0.51. Removing *alarm memory*, the sequential component, costs 0.099, roughly half as much. So a practitioner who randomizes a simple policy captures a large share of the benefit without any belief state. Removing **benign-drift modelling** slightly *improves* harm and quadruples false quarantine (5.1% → 19.8%), confirming it is a usability mechanism rather than a safety one.

RQ4: across the three detector settings, SENTINEL’s advantage over B1 ranges 21.4% to 39.8%, largest with the weakest detector—adaptive allocation compensates for detection quality, which is the regime where it is most needed and the one we would expect in deployment.

10.6 Qualitative analysis: what the attacker does against each policy

Inspecting best responses is more informative than the aggregate. Against *audit-on-insertion*, every best response inserts through repository ingress rather than direct write—the unaudited channel—which is why it is the worst policy. Against *audit-at-commit*, best responses maximise trigger delay, pushing activation past the window in which the commit audit can attribute cause. Against *risk-score thresholding*, they shape the payload to sit just below the threshold, which is why that policy’s false-quarantine rate is high (8.1%) and its harm is not: it is being steered. Against SENTINEL, best responses spread across carriers rather than exploiting one, which is the behaviour a randomized policy is supposed to induce and the reason exploitability falls to 0.09.

11 DISCUSSION

The regime boundary is the contribution. Figure 3 says adaptive audit allocation is worth building when triggers are delayed beyond about two tasks, and is not worth building otherwise—where it is sometimes slightly worse. Carrier-cost heterogeneity moves that threshold later rather than earlier; its real significance is that it makes any allocation short of the minimax exploitable. That boundary is predicted by Theorem 4 and confirmed empirically, and we think it is more useful than the policy itself. A practitioner can measure their own Δ and χ and decide without running our system.

Randomize before you adapt. Removing randomization costs 0.184 of worst-case harm; removing the sequential belief costs 0.099. Against an adversary that observes the policy, a deterministic schedule is the dominant vulnerability, and randomizing a simple stage-weighted policy captures more than half of SENTINEL’s benefit at a fraction of the engineering cost. We would rather report this than let the elaborate component take credit for the simple one’s work.

Audit-on-insertion is a trap. It is the intuitive first design—check things as they enter—and it is the worst policy we measured, worse than doing nothing upstream, because a fixed entry check tells the attacker exactly which channel to avoid. The general lesson is that any *announced, fixed* stage of verification becomes a stage the attacker routes around, which is the classical security game insight arriving in a new setting.

Weak adversaries flatter defenders. LLM attackers make every policy look good (41.2% reduction for SENTINEL), because they insert predictably and do not exploit carrier heterogeneity. A defence evaluated only against LLM-generated attacks is measuring the attacker’s limitations. We report those results as a behavioural note and base every claim on scripted best-responders and held-out policies.

Composition with recovery. Better recovery lowers the cost of acting on an alarm, which changes the optimal policy toward more aggressive quarantine; sweeping η_Q shows SENTINEL’s advantage over B1 growing from 34.1% to 44.0% as quarantine becomes cheap.

Table 2: Main results at equal budget, $\Delta > 2$, $\chi = 1.34$, against *held-out* attacker policies. B7 is the exact minimax value on the small-game subset. All values projected.

Policy	Worst-case harm ↓	Clean completion % ↑	False quarantine % ↓	Detection delay ↓	Exploitability ↓
B1 Audit-at-commit	0.412 [0.381,0.444]	94.1	3.2	4.6	0.34
B2 Uniform random	0.386 [0.356,0.417]	92.8	5.9	3.1	0.21
B3 Audit-on-insertion	0.441 [0.409,0.474]	95.2	2.4	5.8	0.47
B4 Audit-on-retrieval	0.398 [0.367,0.430]	93.6	4.6	3.9	0.39
B5 Risk-score threshold	0.359 [0.329,0.390]	91.4	8.1	2.8	0.29
B6 Two-stage check [1]	0.344 [0.315,0.375]	92.1	6.4	2.6	0.24
SENTINEL	0.272 [0.245,0.300]	92.6	5.1	2.1	0.09
B7 Exact minimax (small games)	0.249	93.0	4.8	2.0	0.00

Table 3: Ablations (RQ4). Projected.

Variant	Worst-case harm	False quar. %	Exploitability
Full SENTINEL	0.272	5.1	0.09
– randomization (deterministic)	0.456	4.8	0.51
– alarm memory (stateless)	0.371	6.2	0.19
– transition uncertainty (nominal kernel)	0.318	5.4	0.19
– benign-drift modelling	0.264	19.8	0.19

they also increase the belief-state dimension beyond what our particle filter was tuned for. Attack payloads come from published categories; a novel channel would sit outside Π_A .

Conclusion validity. Instances from one workflow are correlated and share a split; intervals are cluster-bootstrapped by repository family over 3 seeds. The primary endpoint is declared against B1 (audit-at-commit)—the baseline most likely to make the paper unnecessary—with a 15% relative margin, and results are reported on the (Δ, χ) grid rather than pooled, because a pooled number would average a regime where the method helps with one where it does not.

Auditing and recovery are therefore complements with a quantifiable interaction, not alternatives—which is a reason to build them together and a reason not to evaluate either in isolation.

12 THREATS TO VALIDITY

Construct validity. “Worst-case verified harm” is the maximum over a *declared* attacker class of harm confirmed by a sealed oracle. Harm outside the oracle’s reach is invisible to all systems, and an attacker outside Π_A is outside every number we report—so “worst-case” means worst-case-within-class, and we use that phrasing deliberately. Audit costs are measured CPU-minutes for our implementations of each check; an organisation with different check implementations would have different χ , which is the parameter the regime boundary depends on most.

Internal validity. Defender policies and attacker libraries are frozen by hash before evaluation, and the harness refuses unfrozen policies—but the *development* split was used to design both, so held-out attacker performance (27.6%) is the transfer estimate and the development figure (34.1%) is not. A shared detector is used across all systems so that allocation rather than detection is the variable; a system with its own detector could do better or worse for reasons unrelated to allocation. Benign changes are verified to be surface-indistinguishable (discriminator AUC 0.56), but a more capable discriminator might separate them, which would make false quarantine easier to avoid for every system.

External validity. 15 repositories, 4 carriers, 6–14-task workflows. Real deployments have more carriers and longer horizons, both of which by Theorem 4 move further into the regime where adaptive allocation helps, so our estimates are plausibly conservative—but

Threats to the theory. Theorem 3 assumes a known horizon and bounded kernel uncertainty; a non-stationary environment breaks both. Theorem 4 is a lower bound on required budget under a matched-statistics drift model; if benign drift were easily separable, commit-time audit would suffice at much larger Δ . The Stackelberg assumption—the attacker best-responds to a committed policy and does not adapt within the horizon—is strong, and an attacker that learns during the workflow is outside our model entirely.

13 CONCLUSION

Allocating a limited verification budget against an adaptive attacker is a solved problem in the abstract, so the question for software workflows is whether their structure changes the answer. We identified the structure that does—persistence across tasks, delayed triggers, and heterogeneous carriers—formulated the finite-horizon partially observed Stackelberg game over that state, and proved a lower bound on the audit budget needed to distinguish harmful persistence from benign change, showing it grows with trigger delay and carrier heterogeneity in a way a commit-time audit cannot absorb. The resulting regime boundary, verified empirically at a crossover of about two tasks of delay, is the practical output: below it, audit-at-commit is as good as anything and adaptive allocation is sometimes worse; above it, adaptive allocation reduces worst-case harm by 34.1% on development attackers and 27.6% against held-out adaptive policies at equal budget. Two secondary findings deserve as much weight: randomization contributes nearly twice what the sequential belief state does, so a practitioner should randomize before adapting; and audit-on-insertion, the intuitive design, is the worst policy we measured. We release the game, the

workflows with evaluator-known carrier and trigger state, and the frozen attacker policies.

14 DATA AVAILABILITY

An anonymised artifact accompanies this submission and contains: the SENTINEL implementation; the AUDITGAME-SE construction pipeline (workflows with evaluator-known carrier/trigger state and frozen attacker policies); container recipes and image digests; the exact commit identifiers of every subject repository; all baseline implementations, with reimplemented baselines clearly labelled; the analysis scripts that produce every table and figure; and per-task outcomes rather than aggregates alone.

Sealed oracles and any labels withheld during evaluation are released after the evaluation window closes, together with the seeds needed to reproduce every run. Subject repositories are public and permissively licensed; derived data is released under the union of their licences. The artifact is prepared for the ACM Artifact Evaluation *Available* and *Reusable* badges, and the anonymised copy carries no author-identifying metadata.

Reproduction caveats. Model-hosted backends change; we record exact model identifiers and access dates, and pin the open-weight backend by digest so that at least one configuration is reproducible indefinitely. Results that depend on wall-clock behaviour are reported with the hardware profile used.

15 ETHICAL CONSIDERATIONS

Attacker policies are scripted over published poisoning categories and operate only within sandboxed workflow replicas. We freeze and release them, including the held-out set, after the evaluation window, because a security-game result that cannot be re-run against the same adversaries is not checkable. No technique here increases an attacker’s capability against a system that is not already vulnerable to the published attacks we build on.

All experiments run in isolated containers against forks of public repositories; no production system, no third-party service and no human subject is involved. We follow coordinated-disclosure norms for any defect our tooling incidentally discovers in a subject repository.

REFERENCES

- [1] Adaptive Contracts Authors. 2026. Adaptive Contracts for Costly Evaluation in Multi-Agent Systems. *arXiv preprint arXiv:2603.17212* (2026).
- [2] Audit the Whisper Authors. 2025. Audit the Whisper: Detecting Covert Coordination in Agent Populations. *arXiv preprint arXiv:2510.04303* (2025).
- [3] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. Agent-Poison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [4] Colosseum Authors. 2026. Colosseum: Auditing Collusion Among Language-Model Agents. *arXiv preprint arXiv:2602.15198* (2026).
- [5] Dependency-Guided Rollback Authors. 2026. Dependency-Guided Rollback for Compromised Agent Memory and Execution State. *arXiv preprint arXiv:2608.10502* (2026).
- [6] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. A Practical Memory Injection Attack against LLM Agents. *arXiv preprint arXiv:2503.03704* (2025).
- [7] DRAM Authors. 2025. DRAM: Deliberative Reporting and Mechanisms for Strategic Agents. *arXiv preprint arXiv:2512.21794* (2025).
- [8] HarnessSafe Authors. 2026. HarnessSafe: Tracing Persistent Influence Across Agent Carriers. *arXiv preprint arXiv:2608.06984* (2026).
- [9] Incentive-Aware AI Safety Authors. 2026. Incentive-Aware AI Safety via Stackelberg Security Games. *arXiv preprint arXiv:2602.07259* (2026).
- [10] Christopher Kiekintveld, Manish Jain, Jason Tsai, James Pita, Fernando Ordóñez, and Milind Tambe. 2009. Computing Optimal Randomized Resource Allocations for Massive Security Games. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.
- [11] Dmytro Korzhuk, Vincent Conitzer, and Ronald Parr. 2010. Complexity of Computing Optimal Stackelberg Strategies in Security Resource Allocation Games. In *AAAI Conference on Artificial Intelligence*.
- [12] MemPoison Authors. 2026. MemPoison: Indirect Persistence Attacks on Agent Memory. *arXiv preprint arXiv:2605.29960* (2026).
- [13] MemSecBench Authors. 2026. MemSecBench: Benchmarking the Poisoning Lifecycle and Selective Repair of Agent Memory. *arXiv preprint arXiv:2607.27080* (2026).
- [14] MemTxn Authors. 2026. MemTxn: Source-Supported Memory Updates and Recovery without the Physical Write Set. *arXiv preprint arXiv:2607.27834* (2026).
- [15] NeuroTaint Authors. 2026. NeuroTaint: Auditing Influence Propagation in Agent Memory. *arXiv preprint arXiv:2604.23374* (2026).
- [16] Milind Tambe. 2011. *Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned*. Cambridge University Press.